E-Commerce Microservices on Oracle Kubernetes Engine: Architecture and Implementation

Documentation	Final Project Documentation
Team	Group A
Compartment	`shared-group-a-cmp`
Region	`me-jeddah-1` (Saudi Arabia West – Jeddah)
Repositories	`github.com/Ejada-A`

Team Members

Basel Alaa	Ali Yousef
Yara Adel Ejada	Chris
Abdelrahman Darwish	Habiba Fawzy
Ali Hamad	Kholosy
Ali Tallawy	Peter Kameel
Salma	Beshoy

Summary

For the final week of the Ejada Cloud Build Track internship, Group A took an e-commerce application and modernized it into a microservice architecture deployed on Oracle Kubernetes Engine (OKE), reusing and extending the Terraform module structure built in earlier weeks. This documentation covers both halves of that deliverable: the assessment that turned one application description into four independent microservices, and the Terraform, Helm, and GitOps mechanics used to build, deploy, and continuously update them on OKE.

We split the application into four domains, each owning a distinct slice of the data model: **Auth** (accounts, credentials, roles), **Product** (catalog), **Cart/Orders**, and **Purchase/Payments**. Each domain became its own microservice and its own Git repository, matching a 1:1 mapping with no merging at the service level. The network that hosts them follows the same defense-in-depth shape used in previous weeks: four subnets (API endpoint, load balancer, worker nodes, pods), with only the load balancer subnet public-facing, Internet/NAT/Service gateways routed per tier, and per-resource Network Security Groups generated from a single reusable Terraform pattern rather than hand-written per subnet.

Three architectural points stand out in this build:

- **Domain boundaries were decided by data ownership, not by who is allowed to act on the data.** Admin CRUD on the product catalog was folded into the Product service rather than split into its own service, because the Product service already owns that data.
- **The infrastructure is fully GitOps-driven.** Each microservice's own CI pipeline builds and pushes its container image to OCIR, then triggers a shared workflow that bumps that service's image tag in the Helm chart's `values.yaml`; ArgoCD watches that same file and reconciles the cluster automatically, with no manual `kubectl apply` step in the normal flow.
- **The Terraform network module generalizes the pattern used in earlier weeks.** Subnets, and their ingress/egress rules, are generated from `for_each` over typed maps and flattened rule lists rather than hand-written per subnet, so adding a fifth subnet or a new rule is a data change, not a new resource block.

The compute sizing (node shape, OCPU/memory split) and the database engine per service were left as open implementation decisions in the assessment phase and defaulted to concrete values during the Terraform build, which we note explicitly in Chapter 5 rather than presenting as settled from the outset.

Contents

List of Abbreviations

OCI	Oracle Cloud Infrastructure
OKE	Oracle Kubernetes Engine
OCIR	Oracle Cloud Infrastructure Registry
VCN	Virtual Cloud Network
CIDR	Classless Inter-Domain Routing
NAT	Network Address Translation
NSG	Network Security Group
HPA	Horizontal Pod Autoscaler
PVC	Persistent Volume Claim
CI/CD	Continuous Integration / Continuous Deployment
OCID	Oracle Cloud Identifier
SSH	Secure Shell
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
TCP	Transmission Control Protocol
ICMP	Internet Control Message Protocol
MTU	Maximum Transmission Unit
API	Application Programming Interface
VM	Virtual Machine
IP	Internet Protocol

List of Figures

List of Tables

Chapter 1

Introduction and Scope

For the final week of the Ejada Cloud Build Track internship, teams were asked to take an application running on a native OCI Compute instance, modernize it into a microservice architecture, and migrate it to Oracle Kubernetes Engine (OKE) using the OCI, Terraform, Docker, and Kubernetes skills built up over the preceding weeks. Group A chose an e-commerce application for this build. This documentation covers our combined deliverable: the assessment phase that turned the application's description into a set of microservice boundaries, and the architecture and Terraform/Kubernetes mechanics used to implement, deploy, and continuously update them on OKE.

1.1 Background and Motivation

Splitting a monolithic-style application into independently deployable microservices is a defense-in-depth and scaling strategy at the application layer, in the same way splitting a network into public and private tiers is a defense-in-depth strategy at the infrastructure layer. Each service can be built, scaled, and deployed on its own schedule, and a bug or overload in one domain does not directly take down another. The assignment asked us to go beyond a single-service lab exercise: the resulting architecture had to be reusable across the team, automatically built and deployed through CI/CD, and hosted on infrastructure that already followed the network patterns established in earlier weeks of the program.

1.2 Objective

The primary question this documentation answers is: *what microservice boundaries did we choose for this application, and how are they actually built, containerized, and deployed to OKE end to end?* Two secondary questions follow from it:

- how the assessment phase's data-ownership analysis translated into the final set of services and repositories; and
- how the Terraform network module and the Kubernetes/Helm/ArgoCD deployment pipeline fit together into one automated path from a code push to a running pod.

1.3 Scope and Boundaries

This build targets a single OCI region, `me-jeddah-1`, and a single shared compartment, `shared-group-a-cmp`, hosting one OKE cluster and one Kubernetes namespace, `ecommerce-ns`. The application is split into five Git repositories under the `Ejada-A` GitHub organization: four backend services (`auth-service`, `products-service`, `orders-service`, `payments-service`) and one Next.js frontend (`ecomm-ui`), alongside a separate `Terraform-code` repository holding the infrastructure, Kubernetes manifests, and Helm chart. This documentation does not cover multi-region or multi-availability-domain high availability, load testing under real traffic, or a production compliance review; the compute sizing and per-service database engine were also left as open implementation decisions during the assessment phase, which we address explicitly in Chapter 5.

1.4 Key Stakeholders

Table 1.1: Stakeholders in the Group A final project and this documentation.

Stakeholder	Interest or Role	Relevance
Group A (build team)	Designs, implements, and maintains the four microservices, the frontend, and the Terraform/Helm/ArgoCD deployment pipeline	Collective author of the configuration and application code this documentation describes
Ali Yousef	Group A member; compiled this documentation	Author of this write-up
Ali Hamad	Group A member; credited as Helm chart maintainer and infrastructure/deployment lead in the repository	Primary author of the deployment runbook (`DEPLOYMENT.md`) referenced throughout
Ejada Cloud Build Track	Reviews the assignment deliverable against the stated learning objectives	Target audience for this documentation
Application end users	Interact with the storefront through the public load balancer	Affected by the correctness of the routing and service boundaries described in Chapter 4
Future maintainers	Would extend this architecture, for example by finalizing the per-service database engine or compute sizing	Rely on the implementation approach in Chapter 3 to extend the configuration consistently

1.5 Main Risks and Uncertainties

The assessment phase explicitly left two implementation details open rather than specifying them upfront: the OKE node pool's compute shape and OCPU/memory split, and the specific database engine used per service. Both were resolved with concrete defaults during the Terraform and Kubernetes build, which we document as implemented in Chapter 4 and revisit as an open item in Chapter 5.

1.6 How This Documentation Is Organized

Chapter 2 sets out the domain-ownership analysis behind the four microservice boundaries, the full OKE network design (subnets, gateways, route tables, and security rules), and the application-layer design decisions (scalability, routing, registry, and storage) from the assessment phase. Chapter 3 explains the Terraform and Kubernetes mechanics used to implement that architecture, including the CI/CD and GitOps pipeline that keeps the cluster in sync with each service's own repository. Chapter 4 walks through the architecture as actually deployed: the network, the routing, the autoscaling configuration, and the deployment pipeline. Chapter 5 turns that into our conclusions, and Chapter 6 lists the concrete decisions we would still finalize. The appendix collects a glossary of terms and a command reference.

Chapter 2

Background and Context

In this chapter, we set out the domain-ownership analysis that produced the four microservice boundaries, the full OKE network design (subnets, gateways, route tables, and security rules), the application-layer design decisions (scalability, routing, registry, and storage), and the trade-offs we weighed before settling on the final build.

2.1 Domain-Driven Decomposition

We identified four functional domains directly from the application description: Auth, Product, Cart, and Purchase. A domain is defined by the data it owns, not by who is allowed to touch it, which is why admin CRUD on the product catalog was folded into the Product domain rather than treated as a separate one. Table 2.1 sets out what each domain owns and what it depends on; those dependencies are exactly the calls each microservice makes to the others once deployed.

Table 2.1: Domain ownership and dependencies identified during the assessment phase.

Domain	Owns	Depends on
Auth	User accounts, credentials, roles (admin, user)	Nothing
Product	Catalog: name, description, price, stock, images	Auth (admin role required for create/update/delete; user role can view)
Cart	Cart items, quantities, owning user	Auth (identify user, role must be "user") + Product (existence, stock, current price)
Purchase	Orders, transactions, payment records, receipts	Auth (identify user and role) + Cart (items/quantity being purchased) + Product (confirm final price/stock at purchase time)

All four domains map 1:1 to separate microservices, with no merging at the service level. Cart and Purchase were the borderline call in this mapping, which we discuss in Section 2.6.

2.2 OKE Network Design

The network hosting these services follows Oracle's official OKE network configuration guidance: pods run on worker node hardware, but draw their IP addresses from a separate Pods subnet through a second VNIC attached to each node (VCN-native pod networking), rather than sharing the worker nodes' own address space. Table 2.2 sets out the four subnets this design uses.

Table 2.2: The four-subnet design used for the OKE cluster.

Subnet	Type	Purpose	CIDR	Range
API Endpoint	Public	Kubernetes API access (`kubectl`)	10.0.0.0/28	.0–.15
Load Balancer	Public	Public entry point for the application	10.0.0.16/28	.16–.31
Worker Nodes	Private	Hosts the VMs running all pods	10.0.16.0/20	10.0.16.0–10.0.31.255
Pods	Private	IP addresses for individual pods (VCN-native)	10.0.32.0/19	10.0.32.0–10.0.63.255

The Pods subnet is deliberately larger than the Worker Nodes subnet (a /19 versus a /20), since each node can host many pods and every pod needs its own address from that range.

Gateways

Table 2.3 sets out the three gateways that connect these subnets to the internet and to other OCI services.

Table 2.3: Gateways attached to the VCN.

Gateway	Attached to	Purpose
Internet Gateway	API Endpoint, Load Balancer subnets	Inbound/outbound internet access for public subnets
NAT Gateway	Worker Nodes, Pods subnets	Outbound-only internet access for private subnets
Service Gateway	Worker Nodes, Pods subnets	Private, fast path to OCI services (e.g. OCIR) without using NAT

Route Tables

Each subnet has its own route table, summarized in Table 2.4. The private subnets route Oracle Services Network traffic through the Service Gateway and everything else through the NAT Gateway, so pulling an image from OCIR never touches the NAT path.

Table 2.4: Route tables for each subnet.

Subnet	Destination	Target
API Endpoint (public)	0.0.0.0/0	Internet Gateway
Load Balancer (public)	0.0.0.0/0	Internet Gateway
Worker Nodes (private)	All region services in Oracle Services Network	Service Gateway
Worker Nodes (private)	0.0.0.0/0	NAT Gateway
Pods (private)	All region services in Oracle Services Network	Service Gateway
Pods (private)	0.0.0.0/0	NAT Gateway

Network Security Group Rules

Every ingress and egress rule is attached at the resource level through a Network Security Group rather than a subnet-wide security list (we set out why in Section 2.6). Tables 2.5 through 2.8 list every rule designed for each of the four subnets.

Table 2.5: NSG rules for the Kubernetes API Endpoint subnet.

Direction	Peer	Port	Description
Ingress	Worker Nodes CIDR	TCP 6443	Worker to API endpoint communication
Ingress	Worker Nodes CIDR	TCP 12250	Worker to API endpoint communication
Ingress	Pods CIDR	TCP 6443	Pod to API endpoint communication (VCN-native)
Ingress	Pods CIDR	TCP 12250	Pod to API endpoint communication (VCN-native)
Ingress	Worker Nodes CIDR	ICMP 3,4	Path MTU Discovery
Ingress	Team IPs (optional)	TCP 6443	`kubectl` / client access, not `0.0.0.0/0`
Egress	OCI Services Network	TCP 443	API endpoint to OKE communication
Egress	Pods CIDR	ALL	API endpoint to pod communication (VCN-native)
Egress	Worker Nodes CIDR	ICMP 3,4	Path MTU Discovery
Egress	Worker Nodes CIDR	TCP 10250, ICMP	API endpoint to worker node communication

Table 2.6: NSG rules for the Worker Nodes subnet.

Direction	Peer	Port	Description
Ingress	Worker Nodes CIDR	ALL	Communication between worker nodes
Ingress	Pods CIDR	ALL	Pods on one node reach pods on other nodes
Ingress	API Endpoint CIDR	TCP ALL	API endpoint to worker node communication
Ingress	0.0.0.0/0	ICMP 3,4	Path MTU Discovery
Ingress	Load Balancer CIDR	ALL/10256	LB to kube-proxy health check on worker nodes
Ingress	0.0.0.0/0 (optional)	ALL/30000–32767	NodePort traffic via LB / Network LB
Egress	Worker Nodes CIDR	ALL	Communication between worker nodes
Egress	Pods CIDR	ALL	Worker nodes reach pods on other nodes
Egress	0.0.0.0/0	ICMP 3,4	Path MTU Discovery
Egress	OCI Services Network	TCP ALL	Nodes communicate with OKE
Egress	API Endpoint CIDR	TCP 6443 / 12250	Worker to API endpoint communication
Egress	0.0.0.0/0 (optional)	TCP ALL	General outbound internet access

Table 2.7: NSG rules for the Pods subnet.

Direction	Peer	Port	Description
Ingress	API Endpoint CIDR	ALL	API endpoint to pod communication
Ingress	Worker Nodes CIDR	ALL	Pods on one node reach pods on other nodes
Ingress	Pods CIDR	ALL	Pods communicate with each other, enabling the microservices to call each other
Egress	Pods CIDR	ALL	Pods communicate with each other
Egress	OCI Services Network	ICMP 3,4	Path MTU Discovery
Egress	OCI Services Network	TCP ALL	Pods communicate with OCI services
Egress	API Endpoint CIDR	TCP 6443 / 12250	Pod to API endpoint communication
Egress	0.0.0.0/0 (optional)	TCP 443	Outbound internet access, if needed

Table 2.8: NSG rules for the Load Balancer subnet.

Direction	Peer	Port	Description
Ingress	0.0.0.0/0	TCP 443	Public traffic to the Load Balancer
Egress	Worker Nodes CIDR	ALL/30000–32767	LB forwards traffic to worker nodes
Egress	Worker Nodes CIDR	ALL/10256	LB communicates with kube-proxy health check

SSH (TCP 22) inbound to worker nodes is left as an explicit optional rule for troubleshooting rather than a standing part of the design.

2.3 Security Design

Table 2.9 sets out the security posture the design called for across five areas.

Table 2.9: Security design decisions.

Area	Design decision
Network isolation	Only the Load Balancer subnet is public-facing; Worker Nodes and Pods subnets are private
Access control	NSGs used for granular, per-resource rules rather than broad security lists
API access	Kubernetes API endpoint access restricted to known team IPs, not `0.0.0.0/0`
Secrets	Database credentials and API keys stored as Kubernetes Secrets backed by OCI Vault, never hardcoded in code, images, or Git
IAM	Worker nodes granted a narrowly-scoped dynamic group policy (e.g. OCIR image pulls, Vault reads), not broad compartment-admin rights

Two further constraints shape how traffic and access are handled beyond this table.

- **NAT and Service Gateways split egress traffic by destination.** Worker nodes and pods reach the public internet outbound-only through the NAT Gateway, while traffic bound for other OCI services takes the private Service Gateway path instead, matching Table 2.4.
- **A minimum of two worker nodes is enforced regardless of final sizing.** This gives every deployment's pod replicas real node-level redundancy rather than all landing on a single node.

2.4 Application-Layer Design Decisions

Beyond the network itself, four further decisions were made during the assessment phase: how each service scales, how traffic is routed to it, how its image is registered, and how it stores data.

Scalability: Replica Strategy

Each microservice was designed to use a Horizontal Pod Autoscaler with its own minimum and maximum replica count, rather than one fixed policy for every service, so the deployed environment stays small and low-cost for realistic traffic while still demonstrating per-service scaling design. Table 2.10 sets out the reasoning behind each service's thresholds; we compare this against the deployed autoscaling policy in Section 4.2.

Table 2.10: Per-service HPA design and reasoning.

Service	Min	Max	Reasoning
Auth	1	3	Every request depends on it, so it needs headroom under load
Product	1	3	Constant read traffic from browsing
Cart	1	2	Frequent but tolerant of brief unavailability
Purchase	2	3	Financial data, never allowed to run on a single replica

Traffic Routing

A single Load Balancer and Ingress Controller was designed to give the frontend one public entry point, routed to each service by path, as set out in Table 2.11. We compare this against the deployed ingress rules in Section 4.2.

Table 2.11: Designed path-based routing.

Path	Routed to
`/api/auth/*`	Auth service
`/api/products/*`	Product service
`/api/cart/*`	Cart service
`/api/purchase/*`	Purchase service

Container Registry

Each microservice was designed to have its own OCIR repository, matching the 1:1 microservice mapping: `groupa-auth-app`, `groupa-product-app`, `groupa-cart-app`, and `groupa-purchase-app`.

Storage

Each microservice was designed to own its data through a database-per-service pattern: four separate databases, one per microservice, with no database shared across services, each persisting through its own PVC (Block Volume), following the storage pattern established in earlier weeks. The isolation reasoning mirrors the Cart-versus-Purchase split in Section 2.6: Purchase's permanent financial records must stay isolated from Cart's high-churn, temporary data. The specific database engine per service was left as an implementation detail for the build phase.

2.5 Evidence and Verification Basis

This documentation covers one specific system we designed and built, rather than a literature-based analysis. Table 2.12 summarizes the kinds of evidence referenced throughout, at the point each one supports a claim.

Table 2.12: Evidence types used to support claims in this documentation.

Evidence type	What it confirms	Basis for trust
Assessment and design documentation	The intended domain boundaries, network design, and application-layer decisions	Written during the assessment phase, before implementation began
Terraform configuration (`Terraform-code/`)	The declared network, OKE cluster, and OCIR resources	Committed source, reproducible with `terraform plan`
Kubernetes manifests and Helm chart (`kuber-netes/`, `helm/ecommerce-app/`)	The declared workloads, services, ingress rules, and autoscaling policy actually shipped to the cluster	Committed source, applied via `kubectl`, Helm, or ArgoCD
GitHub Actions workflow files (`.github/workflows/`)	How each service's image is built, tagged, and promoted into the Helm chart	Committed workflow definitions, one per repository
Application source (`server.ts`, models, Next.js routes)	What each service's API actually exposes and how it talks to the others	Read directly from each service repository

2.6 Design Trade-offs Considered

Two decisions in Table 2.1 were not automatic, and are worth setting out explicitly.

Cart versus Purchase. These two were the borderline call in the domain mapping, resolved in favor of keeping them fully separate services rather than merging them. Cart holds temporary, high-churn data, while Purchase holds permanent, immutable financial records that must stay isolated from Cart bugs; the two also have very different traffic and reliability profiles, with Cart hit constantly and Purchase hit rarely but needing to prioritize reliability over speed. In the deployed system, the Cart domain's persistent, validated side is implemented inside the

Orders service, with the shopping cart itself held client-side (in the browser) until checkout; the Purchase domain is implemented as the Payments service.

Network Security Groups versus Security Lists. For this specific design, the two mechanisms would enforce identical traffic, since each subnet holds exactly one resource type. NSGs were used regardless, because they attach to individual resources rather than to a whole subnet, matching Oracle's own recommendation, decoupling security policy from network topology, and remaining reusable if the design changes later, for example if a subnet ever hosts more than one type of resource.

Chapter 3

Implementation Approach

This project reuses and extends the Terraform module structure already built in earlier weeks (the subnet module, the OKE module) rather than rebuilding infrastructure from scratch. In this chapter, we explain the mechanisms that make that configuration reusable across five services and safe to extend, and the pipeline that carries a code change from a service repository into the running cluster.

3.1 Building on the Existing Terraform Foundation

Rather than a manual, console-first build, this project's starting point was the already-validated `network` and `oke` Terraform modules from earlier weeks. The root module (`Terraform-code/main.tf`) wires them together, resolves the latest Oracle Linux image for the chosen compute shape, and provisions one OCIR repository per microservice from a single `for_each`:

Listing 3.1: `Terraform-code/main.tf` – one resource block, five OCIR repositories

```
resource "oci_artifacts_container_repository" "microservices" {
  for_each = local.microservices

  compartment_id = var.compartment_ocid
  display_name   = "shared-group-a-cmp/${each.value}"
  is_public      = false
  is_immutable   = false
}
```

`local.microservices` is the set `auth-service`, `products-service`, `orders-service`, `payments-service`, and `ecomm-ui`. Adding a sixth service later means adding one entry to that set, not writing a new resource block.

Module Interfaces

The `network` and `oke` modules are wired together strictly through typed inputs and outputs, not through any shared state outside Terraform. Figures 3.1 and 3.2 set out each module's contract.

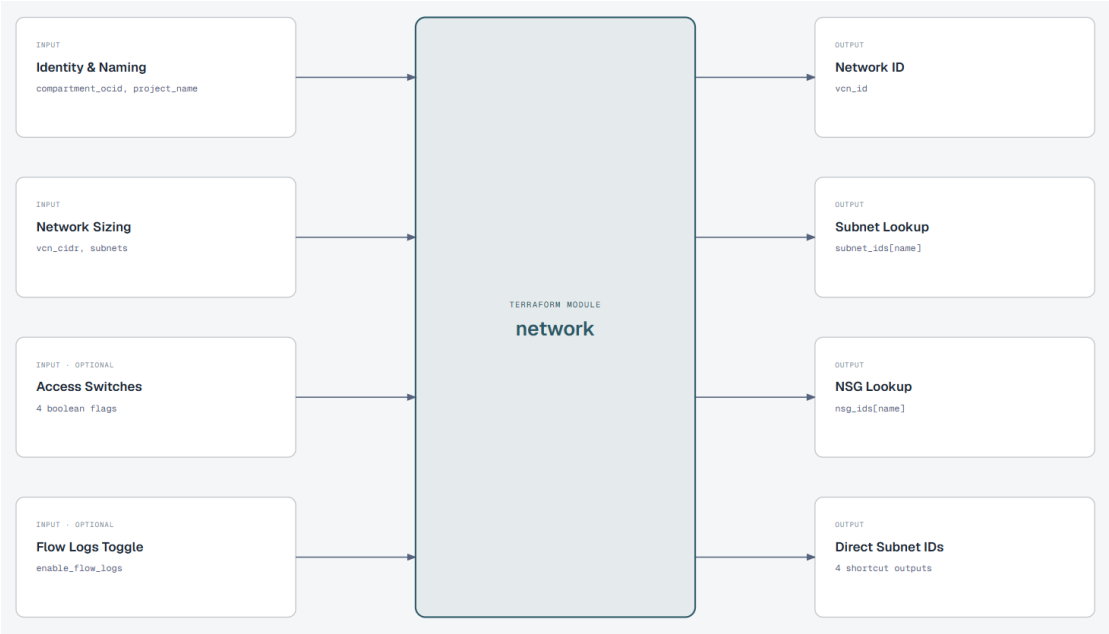

Figure 3.1: Interface of the `network` module. Required inputs fix identity/naming and network sizing; four boolean access switches and a flow-log toggle are optional. Outputs expose the VCN ID plus by-name lookup maps for subnets and NSGs, alongside four shortcut outputs for the individual subnet IDs.

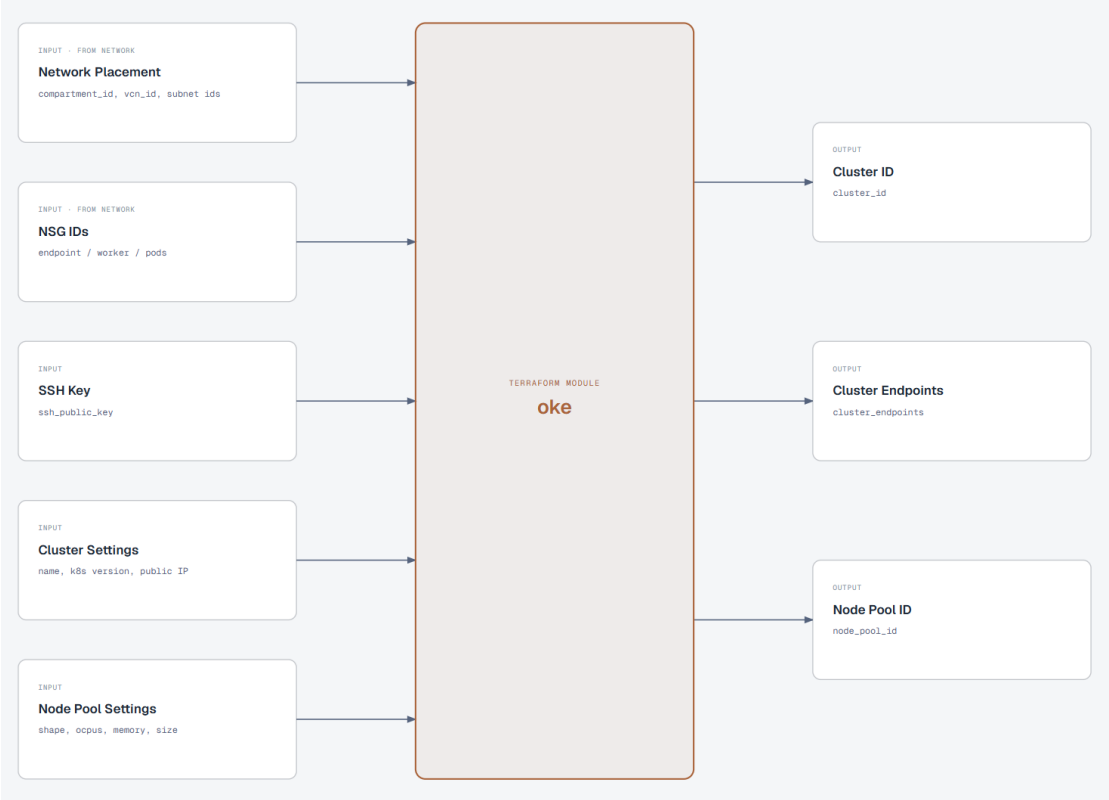

Figure 3.2: Interface of the `oke` module. Two of its inputs, network placement and NSG IDs, are consumed directly from the `network` module's outputs rather than declared independently, which is what makes the root module's `module.oke { vcn_id = module.network.vcn_id, ... }` wiring possible. The remaining inputs (SSH key, cluster settings, node pool settings) are root-level variables. Outputs expose the cluster ID, its API endpoints, and the node pool ID.

This input/output discipline is what lets the root module wire `oke` to `network` with a handful of direct output-to-input references (`vcn_id`, the subnet IDs, the NSG IDs), without either module reaching into the other's internal resources.

3.2 Implementation Method: Configuration and Inputs

Table 3.1 lists the inputs the root Terraform module accepts.

Table 3.1: Selected variables declared in `Terraform-code/variable.tf`.

Variable	Type	Purpose
`compartment_ocid`	string	Target compartment (required, no default)
`region`	string	OCI region (required, no default)
`node_shape`	string	OKE worker node compute shape, default `VM.Standard.A1.Flex`
`node_ocpus`	number	OCPUs per worker node, default `2`
`node_memory_in_gbs`	number	Memory per worker node, default `16`
`node_count`	number	Worker nodes per availability domain, default `2`, floored at `2` regardless of input
`ssh_public_key`	string	SSH public key for optional worker node troubleshooting access
`github_token`, `github_owner`, `github_repo`	string, sensitive	Credentials used to write the OKE cluster OCID back into GitHub Actions secrets

3.3 Configuration Framework: Reducing Repetition

The network module's NSG rules are the clearest example of avoiding repetition in this configuration. Every ingress and egress rule, for every subnet, is declared once as data in `modules/network/locals.tf`, then flattened into a single map keyed by `"<subnet>-<index>"` so one `for_each` can generate every rule resource:

Listing 3.2: `modules/network/locals.tf` – flattening per-subnet rule lists into one for_each-able map

```
ingress_rules_flat = merge([
  for nsg_key, rules in local.ingress_by_nsg : {
    for idx, rule in rules :
    "${nsg_key}-${idx}" => merge(rule, { nsg_key = nsg_key })
  }
]...)
```

Adding a seventh ingress rule to any subnet means adding one object to the relevant list in `ingress_by_nsg`; the resource block that actually creates `oci_core_network_security_group_security_rule` resources never changes. The same `for_each` pattern is used for the four subnets themselves, each built from a

typed `map(object({cidr_block, is_public}))` variable rather than four separate resource blocks.

3.4 Change-Safety: Promoting One Service at a Time

Because each microservice has its own repository, its own CI pipeline, and its own entry in the Helm chart's `values.yaml`, changing one service's image does not require touching Terraform or the other four services. Table 3.2 sets out how a change actually reaches the cluster.

Table 3.2: How a code change in one service reaches the running cluster.

Step	What happens
Push to `main` in a service repository	That repository's own `ci.yml` builds the Docker image and pushes it to OCIR, tagged with both the GitHub Actions run number and `latest`
Reusable `update-helm.yml` workflow	Checks out `Terraform-code`, converts the service's dash-case name to the matching `values.yaml` key, and uses `yq` to bump only that service's `image.tag`
Commit and push to `Terraform-code`	The updated `values.yaml` is committed back to the Helm chart's repository
ArgoCD reconciliation	ArgoCD's automated sync (prune and self-heal enabled) detects the change and rolls out the new image for that one deployment

Because `update-helm.yml` only ever edits one service's `image.tag` key, promoting `payments-service` cannot accidentally change what image `auth-service` is running. Figure 3.3 shows the same sequence as a single left-to-right path.

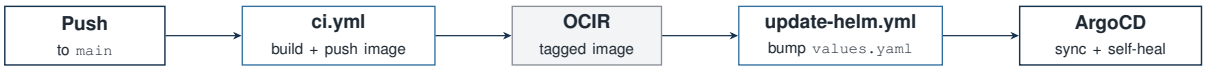

Figure 3.3: The image-promotion pipeline as one path: a push to a single service's repository ends, four steps later, in ArgoCD rolling out only that service's new image.

3.5 Container Image Build and Health Checks

Every backend service is built from the same two-stage Dockerfile pattern, shown here for `auth-service`:

Listing 3.3: `auth-service/Dockerfile`

```
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci || npm install
COPY . .

FROM node:20-alpine AS runner
WORKDIR /app
ENV NODE_ENV=production
COPY --chown=node:node --from=builder /app /app
USER node
EXPOSE 5001

HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \
  CMD wget --no-verbose --tries=1 --spider http://localhost:5001/health
      || exit 1

CMD ["npm", "start"]
```

The builder stage installs dependencies and copies source; the runner stage copies only the finished `/app` directory forward and switches to the non-root `node` user before starting the process, so the container never runs as root. Each service exposes its own `/health` endpoint (a plain 200 response, no dependency checks), which both the Docker-level `HEALTHCHECK` and the Kubernetes liveness and readiness probes target.

The four backend services (`auth-service`, `products-service`, `orders-service`, `payments-service`) use the same probe timing in their Kubernetes manifests, set out in Table 3.3.

Table 3.3: Liveness and readiness probe timing, common to the four backend services.

Probe	**Initial delay / period / timeout**	**Failure threshold**
Liveness (`GET /health`)	15s initial delay, every 10s, 5s timeout	3 consecutive failures restart the container
Readiness (`GET /health`)	5s initial delay, every 5s, 3s timeout	3 consecutive failures remove the pod from Service routing

The readiness probe's shorter delay and interval mean a pod is pulled out of rotation quickly if it stops responding, while the liveness probe's longer thresholds avoid restarting a container that is merely slow to start.

3.6 Records as the Source of Truth

Two records anchor this system, at two different layers. Terraform's own state file is the source of truth for the infrastructure it manages: the VCN, subnets, NSGs, the OKE cluster, and the OCIR repositories. Above that, once the cluster exists, the Helm chart's `values.yaml` inside `Terraform-code` is the source of truth for *which image tag each workload is currently running*, and ArgoCD's own sync status is the record of whether the live cluster actually matches that file. This is the core GitOps property the pipeline in Section 3.4 relies on: the desired state lives in Git, not on any one operator's machine.

3.7 Assumptions and Limitations

Two decisions were left open in the assessment phase and resolved with concrete defaults during the Terraform build rather than being specified upfront: the OKE node pool's compute shape and OCPU/memory split defaulted to `VM.Standard.A1.Flex` at 2 OCPUs and 16 GB of memory, and the specific database engine per service was resolved by running a single shared MongoDB instance for all five services rather than one database per service. We revisit both as open items in Chapter 5. Separately, the root module's IAM policy resources (a dynamic group and policy that would let OKE worker nodes pull images and read secrets without a static auth token) are present in `main.tf` but commented out, with an inline note that the team did not have tenancy-level access to create dynamic groups in this environment; an OCIR auth token is used instead.

3.8 Environment Teardown

The deployment runbook (`Terraform-code/DEPLOYMENT.md`) documents teardown in dependency order, uninstalling the application before destroying the infrastructure that hosts it:

Listing 3.4: Teardown sequence from DEPLOYMENT.md

```
helm uninstall ecommerce -n ecommerce-ns
kubectl delete -f ./Project/Terraform-code/kubernetes/

cd /home/ali_hamad/         /Project/Terraform-code
           destroy
```

The application layer (Helm release or raw manifests) is removed first, then the underlying OKE cluster and network are destroyed through Terraform.

Chapter 4

Deployment Results and Verification

In this chapter, we present the system as it is actually configured to deploy, based on the committed Terraform configuration, Kubernetes manifests, Helm chart, and CI/CD workflows in the `Ejada-A` repositories, rather than as designed on paper.

4.1 The Five Services, as Deployed

Figure 4.1 is the team's own architecture diagram for the deployed system: the full path from a developer or shopper, through the public subnets, into the private worker nodes and pods, and out to OCIR for image pulls.

Figure 4.2 narrows in on the application layer: the Ingress Controller's path-based routing to each of the five workloads, and the shared MongoDB instance every one of them, including the frontend itself, connects to directly.

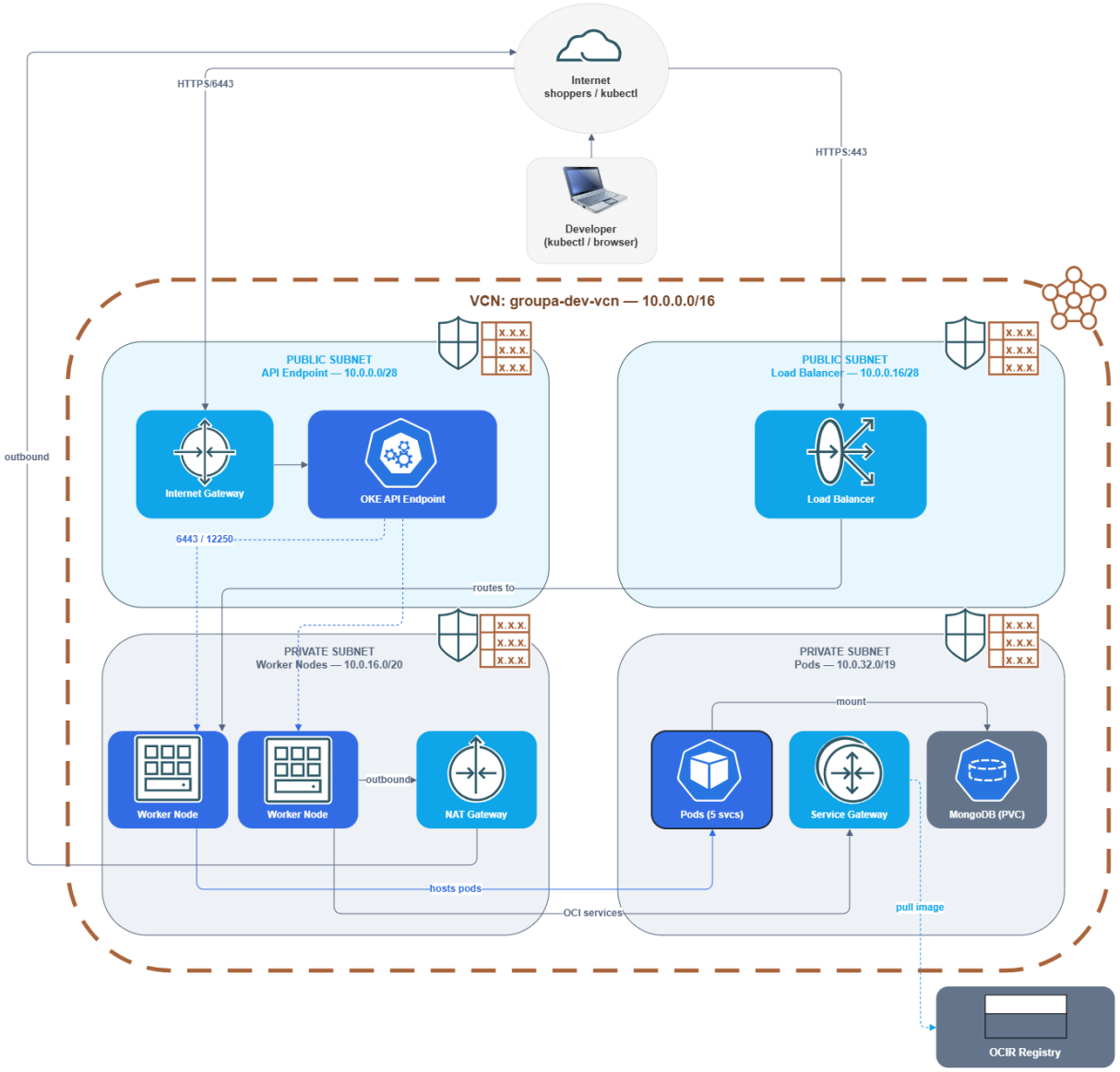

Figure 4.1: Deployed architecture inside VCN `groupa-dev-vcn` (`10.0.0.0/16`). A developer reaches the OKE API endpoint over `HTTPS/6443` through the Internet Gateway; shoppers reach the Load Balancer over `HTTPS:443`. Both public subnets route into the private Worker Nodes subnet (`10.0.16.0/20`, two nodes, egress via NAT Gateway) and Pods subnet (`10.0.32.0/19`, five services plus the shared MongoDB PVC), which reaches OCI services, including pulling images from the OCIR Registry, through the Service Gateway rather than the NAT path.

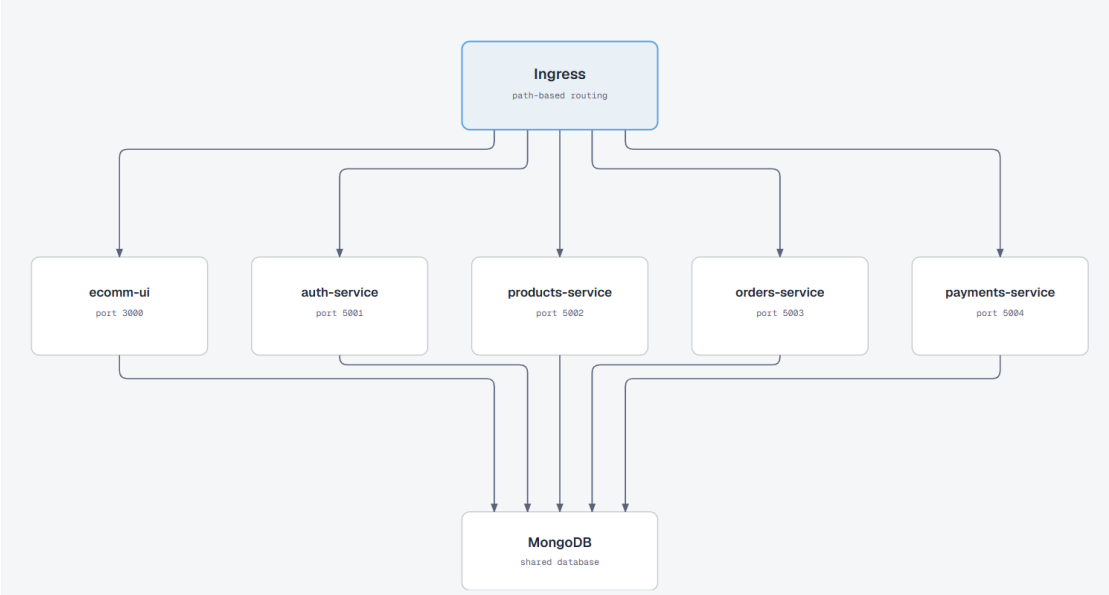

Figure 4.2: Application-layer access structure. The Ingress routes by path to `ecomm-ui` and each of the four backend services by their internal ClusterIP service name and port; every workload, including `ecomm-ui` itself, holds its own connection to the single shared MongoDB instance.

This corrects a simplification worth flagging: `ecomm-ui` is not a pure stateless proxy in front of the four backend services. Most of its API routes do proxy to the corresponding backend service, but it also holds its own direct MongoDB connection (`src/lib/db.ts`) alongside that proxying.

Each backend service builds and pushes to its own OCIR repository under `jed.ocir.io/axkjllkftxfz/shared-group-a-cmp/`, matching the domain boundaries set out in Table 2.1: `auth-service`, `products-service`, `orders-service`, `payments-service`, and `ecomm-ui`. The one-repository-per-service pattern matches the registry design in Section 2.4, though the repository names themselves moved from the design phase's `groupa-*-app` convention to the plural, per-service names above.

4.2 Traffic Routing and Autoscaling

The ingress resource performs path-based routing from the single public host to each backend service, as set out in Table 4.1; compare this against the routing designed during the assessment phase in Table 2.11.

Table 4.1: Path-based routing rules deployed in the ingress resource.

Path	Routed to	Port
/	ecomm-ui	80
/api/auth	auth-service	5001
/api/products	products-service	5002
/api/orders	orders-service	5003
/api/payments	payments-service	5004

Every workload, including the frontend, scales independently through its own Horizontal Pod Autoscaler, deployed with a uniform policy across all five: a minimum of 2 replicas and a maximum of 5, scaling on 75% CPU utilization (the raw Kubernetes manifests additionally target 80% memory utilization). This keeps every service at a minimum of two replicas for redundancy while capping the maximum to bound cost, in place of the per-service minimum and maximum designed during the assessment phase (Table 2.10).

Table 4.2 lists the CPU and memory each pod requests and is capped at; the four backend services all use the identical values, while the frontend and the shared MongoDB instance are sized larger.

Table 4.2: Per-pod resource requests and limits.

Workload	CPU request / limit	Memory request / limit
auth-service, products-service, orders-service, payments-service	100m / 500m	128Mi / 256Mi
ecomm-ui	200m / 1000m	256Mi / 512Mi
mongodb	200m / 1000m	512Mi / 1Gi

These per-pod values are a separate sizing decision from the node pool's own compute shape in Section 3.7: the node pool sizing bounds how much total CPU and memory each worker node has to offer, while these requests and limits bound how much of that capacity any single pod can claim. The HPA scales replica *count* against these per-pod requests, not against the node pool directly.

4.3 The GitOps Deployment Pipeline

The `argocd/ecommerce-app.yaml` Application resource, once applied, points ArgoCD at the `helm/ecommerce-app` path of the `Terraform-code` repository:

Listing 4.1: `Terraform-code/argocd/ecommerce-app.yaml`

```
spec:
  source:
    repoURL: 'https://github.com/Ejada-A/Terraform-code.git'
    path: helm/ecommerce-app
  destination:
    namespace: ecommerce-ns
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

With `selfHeal` enabled, a manual `kubectl edit` against a live Deployment would be reverted on the next reconciliation; the Helm chart in Git is the only path ArgoCD treats as authoritative, matching the source-of-truth split described in Section 3.6.

4.4 Verification Performed

This documentation was compiled from the repositories under the `Ejada-A` GitHub organization rather than from a live session against the running cluster. What we verified directly:

- every ingress path, port, and HPA threshold in Section 4.2 was read directly from the committed Kubernetes manifests and the Helm chart's templates and `values.yaml`;
- the image-promotion pipeline in Section 3.4 was traced through each service's own `ci.yml` and the shared `update-helm.yml` workflow; and
- the domain-to-service mapping in Table 2.1 was confirmed against each service's own `server.ts` and data models.

We have not independently confirmed these against a live `kubectl get pods` or a request against the public load balancer at the time of writing this documentation.

4.5 Open Implementation Decisions

Two decisions were explicitly left open during the assessment phase and resolved with a concrete default during the Terraform and Kubernetes build:

- the OKE node pool's compute shape and OCPU/memory split, defaulted to `VM.Standard.A1.Flex` at 2 OCPUs and 16 GB; and
- the database engine per service, resolved as one shared MongoDB instance for all five services rather than a separate database per service.

4.6 Key Unknowns

This build does not cover multi-region or multi-availability-domain high availability, an HTTPS listener on the ingress, or load testing against the deployed autoscaling thresholds; all three are out of scope for this documentation rather than open failures.

Chapter 5

Conclusions

5.1 Main Conclusion

The system we built implements the four-domain microservice split identified in the assessment phase as a 1:1 mapping to five Git repositories (four backend services plus the frontend), deployed to OKE through a fully GitOps-driven pipeline: each service's own CI builds and pushes its image, a shared workflow promotes that image's tag into the Helm chart, and ArgoCD reconciles the cluster automatically from that chart. That answers the objective set out in Section 1.2 directly: the data-ownership analysis from the assessment phase is traceable, service by service, into the repositories and deployment configuration described in Chapter 4.

5.2 Strongest Supporting Evidence

Three pieces of evidence carry the most weight for this conclusion:

- **The domain-ownership table matches the repository split exactly.** Each domain in Table 2.1 corresponds to exactly one service repository and one OCIR image, with no domain spread across two repositories.
- **The Terraform network module generalizes cleanly.** The same `for_each`-over-flattened-rules pattern used for four subnets in earlier weeks was reused here without modification to the resource blocks themselves.
- **The GitOps pipeline is fully wired, end to end, in committed configuration.** Every step from a code push to a cluster reconciliation, in Section 3.4, is defined in a workflow or manifest file we read directly, not asserted from memory.

5.3 Biggest Uncertainty

The largest open item is that two implementation decisions, the OKE node pool's compute sizing and the database engine per service, were left unspecified during the assessment phase and resolved with a working default during the build rather than a deliberate choice made upfront. Reducing that uncertainty means deliberately revisiting both defaults against the application's actual resource usage, rather than leaving them at whatever the Terraform and Helm defaults happened to be.

5.4 Strategic Implications and Recommendation

The current architecture is appropriate as a working implementation of the assessment phase's domain boundaries. Before treating the compute sizing and shared-database decisions as final, we would revisit both deliberately, each already a one-line change in the existing configuration rather than a restructuring: the node shape and OCPU/memory split are Terraform variables, and the database choice is a matter of provisioning a separate database per service and repointing the relevant `MONGODB_URI` value.

5.5 Confidence Level and Final Judgement

Our confidence in the architecture and deployment-pipeline description in Chapter 4 is **high**: every claim traces back to a specific file in a specific repository under the `Ejada-A` GitHub organization. Our confidence in how the system behaves once actually running is **lower**, since this documentation was compiled from the repositories themselves rather than from a live session against the deployed cluster, as noted in Section 4.4.

Chapter 6

Recommendations and Next Steps

6.1 Further Testing Needed

Two follow-up exercises would materially reduce the uncertainty named in Section 5.3:

- a live verification pass against the running cluster (`kubectl get pods`, a request against the public load balancer) to confirm the deployment behaves as the committed configuration describes; and
- a basic load test against the deployed autoscaling thresholds, to see how the uniform 2–5 replica, 75% CPU policy behaves under realistic traffic for each of the five services.

6.2 Data to Validate

The item most likely to change first is the compute sizing and database-engine defaults named in Section 4.5, since both were explicitly left open rather than deliberately chosen. Anyone extending this configuration should re-read `variable.tf` and the Helm chart's `values.yaml` directly rather than trust the defaults quoted in this documentation.

6.3 Recommended Actions

1. **Finalize the node pool sizing.** Revisit `node_shape`, `node_ocpus`, and `node_memory_in_gbs` against the application's actual observed resource usage rather than the current defaults.
2. **Decide the per-service database question deliberately.** Either confirm the shared MongoDB instance as the intended long-term design, or provision a separate database per service as originally scoped in the assessment phase.

3. **Add an HTTPS listener.** The current ingress is HTTP only; a production-facing version of this build would add a managed certificate.

6.4 Monitoring Indicators

Table 6.1: Indicators worth monitoring as this deployment matures.

Indicator	Why It Matters	Review Frequency
Per-service HPA CPU/memory utilization vs. the 75%/80% thresholds	Signals whether the uniform autoscaling policy fits each service's actual traffic shape	After any load test
ArgoCD sync status for `ecommerce-app`	Confirms the running cluster still matches the Helm chart in `Terraform-code`	Continuous, via ArgoCD's own dashboard
MongoDB PVC utilization (10Gi, shared)	Signals when the single shared database instance is approaching a capacity or contention limit	Monthly, or after any product-catalog seeding

6.5 Open Questions

- whether the shared MongoDB instance should be split into one database per service before this architecture is reused beyond the internship context; and
- whether the OKE IAM dynamic group policy, currently commented out in `main.tf` for lack of tenancy access, should be revisited once broader access is available.

Appendix

.1 Glossary

Table 2: Terms used throughout this documentation.

Term	Meaning
OKE	Oracle Kubernetes Engine: OCI's managed Kubernetes service.
OCIR	Oracle Cloud Infrastructure Registry: the container image registry each service pushes to.
NSG	Network Security Group: a firewall attached to individual resources rather than a whole subnet.
HPA	Horizontal Pod Autoscaler: scales a Deployment's replica count between a minimum and maximum based on resource utilization.
BFF	Backend for Frontend: here, `ecomm-ui`'s own API routes, most of which proxy to the corresponding backend service, though `ecomm-ui` also holds its own direct MongoDB connection alongside that proxying.
GitOps	A deployment model where the desired state of the cluster lives in a Git repository, and an operator (ArgoCD) continuously reconciles the live cluster to match it.
Helm chart	A templated bundle of Kubernetes manifests, parameterized by a single `values.yaml` file.
ArgoCD	The GitOps controller that watches the Helm chart in `Terraform-code` and applies it to the cluster.
PVC	Persistent Volume Claim: a request for durable storage, here backing the shared MongoDB deployment.
ClusterIP	A Kubernetes Service type reachable only from inside the cluster, used for internal service-to-service calls.
kube-proxy	The per-node Kubernetes component that implements Service-to-Pod routing; the Load Balancer's health check queries it directly on each worker node.
Liveness/ readiness probe	Kubernetes health checks against a pod: a failed liveness probe restarts the container, a failed readiness probe removes it from Service routing without restarting it.

.2 Command Reference

Table 3: Commands referenced in this documentation, drawn from `Terraform-code/DEPLOYMENT.md`.

Command	Used for
`terraform init / plan / apply`	Provision the VCN, subnets, OKE cluster, and OCIR repositories
`oci ce cluster create-kubeconfig`	Generate a local `kubectl` context for the OKE cluster
`kubectl apply -f ./kubernetes/`	Apply the raw Kubernetes manifests directly (the non-GitOps deployment path)
`helm install ecommerce ./helm/ecommerce-app -n ecommerce-ns`	Deploy the application through the Helm chart
`kubectl rollout restart deployment -n ecommerce-ns`	Restart all microservice deployments in the namespace
`helm uninstall ecommerce -n ecommerce-ns`	Remove the Helm-managed application before infrastructure teardown
`terraform destroy`	Tear down the OKE cluster and network once the application layer is removed

References

This documentation covers one specific system we designed and built, rather than a literature-based analysis. We did not cite external sources, third-party claims, or published works. Every factual claim is instead traceable to the committed Terraform configuration, Kubernetes manifests, Helm chart, CI/CD workflows, or application source under the `Ejada-A` GitHub organization, following the evidentiary basis set out in Section 2.5. We kept `References/myref.bib` in the repository for template compatibility, but it intentionally contains no entries, so no citation keys appear in this documentation.